Memory-Efficient Versioned File Indexer

CS253 Assignment 1 - Report

Joel Bansal
Roll Number: 230509

March 2026

Contents

1 Introduction **2**

2 Design and Approach **2**
2.1 Architecture Overview 2
2.2 Execution Pipeline 2
2.3 Data Structure 2

3 Description of Classes **3**
3.1 `Config` 3
3.2 `VersionedIndex` 3
3.3 `Tokenizer` 3
3.4 `BufferedReader` 3
3.5 `Query (Abstract Base Class)` 3
3.6 `WordQuery` 3
3.7 `TopKQuery` 3
3.8 `WordDiffQuery` 4
3.9 `QueryProcessor` 4

4 Fixed-Size Buffer Processing **4**
4.1 Buffer Allocation 4
4.2 Handling Tokens Split Across Chunk Boundaries 4
4.3 Memory Complexity 5

5 Design Requirements Implementation **5**
5.1 Inheritance - Abstract Base Class + Derived Classes 5
5.2 Runtime Polymorphism - Virtual Functions and Dynamic Dispatch 5
5.3 Function Overloading 5
5.4 User-Defined Templates 6
5.5 Exception Handling (`try / catch / throw`) 6
5.6 Smart Pointers (`std::unique_ptr`) 7

6 Assumptions and Observations **7**
6.1 Assumptions 7
6.2 Execution Results 7

1 Introduction

The objective is to build a **word-level frequency indexer** that processes large text log files incrementally using a **fixed-size buffer** (256–1024 KB), without loading the entire file into memory.

- A *word* is a maximal contiguous sequence of alphanumeric characters (`std::isalnum`).
- All matching is **case-insensitive** - words are lowercased before indexing.
- Each file is associated with a user-provided *version* label; multiple versions are supported in one run.

Three query types are supported:

1. **word** - returns frequency of a given word in a version
2. **top** - returns top-K most frequent words in a version (descending order)
3. **diff** - returns frequency difference of a word between two versions ($count_{v2} - count_{v1}$)

For example, the text `error debug error info` produces the index:

```
error -> 2,  debug -> 1,  info -> 1
```

2 Design and Approach

2.1 Architecture Overview

The solution is implemented in a single file `230509_joel.cpp` using 9 classes organized into three layers:

1. **Configuration Layer** - `Config`
2. **Processing Layer** - `BufferedReader`, `Tokenizer`, `VersionedIndex`
3. **Query Layer** - `Query` (abstract), `WordQuery`, `TopKQuery`, `WordDiffQuery`, `QueryProcessor`

2.2 Execution Pipeline

1. `Config` parses and validates all CLI arguments.
2. `QueryProcessor` owns all objects. It calls `BufferedReader::readFile()` to read the input file(s) in fixed-size chunks.
3. Each chunk is passed to `Tokenizer::process()`, which extracts lowercase alphanumeric tokens and calls `VersionedIndex::updateFreq()` for each token.
4. After indexing, `QueryProcessor` creates the appropriate `Query` subclass via `std::make_unique` and calls `q->execute()` (virtual dispatch).
5. The `QueryProcessor` destructor prints buffer size and execution time.

2.3 Data Structure

`VersionedIndex` stores a two-level map:

```
std::map<std::string, std::unordered_map<std::string, int>>
    frequencyMap;
```

- Outer `std::map`: $O(\log V)$ lookup by version name
- Inner `std::unordered_map`: amortized $O(1)$ per word insertion/lookup

3 Description of Classes

3.1 Config

Parses `argv[]` in the constructor. Validates:

- Required flags per query type (`--file`, `--version`, `--query`, etc.)
- Buffer size $\in [256, 1024]$ KB
- $k > 0$ for top query; non-empty `--word` for word/diff queries

Throws `std::runtime_error` on any invalid or missing argument. Provides getter methods for all parsed values.

3.2 VersionedIndex

Stores one `unordered_map<string,int>` per version inside a `std::map`. Key methods:

```
void updateFreq(const std::string& word, const std::string& version);
int getFrequency(const std::string& word, const std::string& v) const;
const std::unordered_map<std::string,int>& getVersionData(const std::
    string& v) const;
bool versionExists(const std::string& version) const;
```

3.3 Tokenizer

Scans raw text character-by-character. Accumulates alphanumeric characters (lowercased via `std::tolower`) into a token string; on encountering a non-alphanumeric character, flushes the token to `VersionedIndex::updateFreq()`. Provides two overloaded `process()` methods (see Section 5.3).

3.4 BufferedReader

Reads the file in fixed-size chunks using `std::ifstream::read()` and delegates each chunk to `Tokenizer::process()`. Handles tokens split across chunk boundaries (see Section 4).

3.5 Query (Abstract Base Class)

Declares a pure virtual method and a virtual destructor:

```
class Query {
public:
    virtual ~Query() {}
    virtual void execute() = 0;
};
```

3.6 WordQuery

Derives from `Query`. Calls `VersionedIndex::getFrequency()` and prints version name and count using the global `printOutput<T>` template.

3.7 TopKQuery

Derives from `Query`. Retrieves the full frequency map for a version, passes it to the `getTopK<MapType>` template function which sorts entries by value (descending) and truncates to K. Prints results using `printOutput<T>`.

3.8 `WordDiffQuery`

Derives from `Query`. Retrieves frequency maps for both versions, looks up the target word in each, and prints individual counts and the difference ($count_{v2} - count_{v1}$) using `printOutput<T>`.

3.9 `QueryProcessor`

Owns `Config&`, `VersionedIndex`, `Tokenizer`, and `BufferedReader`. The constructor records `start_time` via `std::chrono::high_resolution_clock`. The `run()` method:

1. Reads file(s) and builds the index
2. Creates the correct `Query` subclass via `std::make_unique`
3. Calls `q->execute()` (virtual dispatch)

The destructor computes elapsed time and prints buffer size and execution time.

4 Fixed-Size Buffer Processing

4.1 Buffer Allocation

- Buffer size b is set once from `--buffer` (default 512 KB), validated to $[256, 1024]$ KB.
- Allocated via `std::unique_ptr<char[]> buffer(new char[b])` - single allocation, auto-freed.
- Each `ifstream::read(buffer.get(), b)` reads at most b bytes per iteration.

4.2 Handling Tokens Split Across Chunk Boundaries

When a chunk boundary falls mid-word, the trailing partial token must be carried forward. The algorithm:

```
std::string prev_left_string = "";
std::unique_ptr<char[]> buffer(new char[bufferSizeInBytes]);

while (file.read(buffer.get(), bufferSizeInBytes) || file.gcount() > 0)
    {
    auto bytesRead = file.gcount();
    if (bytesRead == 0) break;

    std::string curr_chunk(buffer.get(), bytesRead);
    std::string combined = prev_left_string + curr_chunk;

    // scan backward to find last non-alphanumeric char
    size_t last_boundary = combined.size();
    for (int i = combined.size()-1; i >= 0; i--) {
        if (!std::isalnum((unsigned char)combined[i])) {
            last_boundary = i;
            break;
        }
    }

    // process complete tokens only
    token.process(combined.substr(0, last_boundary), version);
    // carry partial trailing token to next chunk
    prev_left_string = combined.substr(last_boundary);
}
// flush any remaining partial token after EOF
if (!prev_left_string.empty())
    token.process(prev_left_string, version);
```

Invariant: `prev_left_string` contains at most one partial token (entirely alphanumeric suffix). No token is ever lost or double-counted.

4.3 Memory Complexity

$$M_{total} = \underbrace{b}_{fixedbuffer} + \underbrace{O(W \cdot \bar{\ell})}_{index:\ W\ uniquewords, avglength\ \bar{\ell}} + \underbrace{O(b)}_{tempstrings}$$

Memory is $O(W)$, independent of file size F.

5 Design Requirements Implementation

5.1 Inheritance - Abstract Base Class + Derived Classes

`Query` is an abstract base class with a pure virtual function. Three classes inherit from it:

```
class Query {
public:
    virtual ~Query() {}
    virtual void execute() = 0;   // pure virtual
};

class WordQuery     : public Query { void execute() override; };
class TopKQuery     : public Query { void execute() override; };
class WordDiffQuery : public Query { void execute() override; };
```

This satisfies the requirement of at least one abstract base class and two derived classes.

5.2 Runtime Polymorphism - Virtual Functions and Dynamic Dispatch

In `QueryProcessor::run()`, the concrete query type is selected at runtime based on user input, and `execute()` is called through a base-class pointer:

```
std::unique_ptr<Query> q;

if (query == "word")
    q = std::make_unique<WordQuery>(w, v, vi);
else if (query == "top")
    q = std::make_unique<TopKQuery>(cfg.getVersionName(), cfg.getK(),
        vi);
else if (query == "diff")
    q = std::make_unique<WordDiffQuery>(cfg.getWord(), cfg.getVersion1
        (),
                                        cfg.getVersion2(), vi);

q->execute();   // resolved at runtime via vtable lookup
```

5.3 Function Overloading

Two pairs of overloaded functions with different parameter lists:

(1) `Tokenizer::process()`:

```
void process(const std::string& text, const std::string& version);
void process(const std::string& text, const std::string& version,
             size_t chunkOffset);   // extra param for debug logging
```

(2) `QueryProcessor::log()`:

```
void log(const std::string& msg);
void log(const std::string& msg, double val);
```

5.4 User-Defined Templates

Two function templates are defined:

(1) `printOutput<T>` - global template for type-generic formatted output:

```
template <typename T>
void printOutput(const std::string& label, const T& value) {
    std::cout << label << ": " << value << '\n';
}

// instantiated with different types:
printOutput("Version", v1);                          // T = std::string
printOutput("Count", count);                         // T = int
printOutput("Execution Time (s)", elapsed.count());  // T = double
```

Used in all three query `execute()` methods and in the `QueryProcessor` destructor.

(2) `getTopK<MapType>` - template parameterized on any map-like container:

```
template<typename MapType>
std::vector<std::pair<std::string, typename MapType::mapped_type>>
getTopK(const MapType& m, int k) {
    std::vector<std::pair<std::string,
        typename MapType::mapped_type>> vec(m.begin(), m.end());
    std::sort(vec.begin(), vec.end(),
        [](const auto& a, const auto& b) { return a.second > b.second;
            });
    if (k < (int)vec.size()) vec.resize(k);
    return vec;
}
```

5.5 Exception Handling (`try` / `catch` / `throw`)

Exceptions are thrown and caught at multiple levels:

```
// Config: invalid buffer size
if (bufferSizeInKB < 256 || bufferSizeInKB > 1024)
    throw std::runtime_error("Buffer Size must be between 256KB to 1024
        KB");

// VersionedIndex: non-existent version
if (frequencyMap.find(v) == frequencyMap.end())
    throw std::runtime_error("Version not found.");

// BufferedReader: file open failure
if (!file.is_open())
    throw std::runtime_error("Failed to open file: " + path);

// main(): catch hierarchy
try {
    Config cfg(argc, argv);
    QueryProcessor qp(cfg);
    qp.run();
}
```

```
catch (const std::invalid_argument& e) { /* stoi failed */ }
catch (const std::out_of_range& e)     { /* number overflow */ }
catch (const std::exception& e)        { /* all runtime errors */ }
catch (...)                            { /* unknown */ }
```

5.6 Smart Pointers (`std::unique_ptr`)

Used in two places to ensure automatic deallocation and exception safety:

```
// fixed buffer -- freed automatically when readFile() returns
std::unique_ptr<char[]> buffer(new char[bufferSizeInBytes]);

// query object -- safe even if execute() throws
std::unique_ptr<Query> q = std::make_unique<WordQuery>(...);
q->execute();
```

6 Assumptions and Observations

6.1 Assumptions

- Word delimiters: any character where `std::isalnum()` returns false
- Buffer size is fixed at startup; no dynamic resizing
- Version names are unique per run; re-using a name merges counts
- Input files use byte-compatible encoding (ASCII / UTF-8 alphanumeric subset)

6.2 Execution Results

The screenshot below shows the compilation and execution of all query types on the provided datasets:

The observed results from the execution are summarized below:

Query	Word	Buffer	Result	Time (s)
word	`error`	256 KB	Count: 605 079	1.918
word	`DEVOPs`	256 KB	Count: 1 209 558	1.810
word	`DEVOPS`	256 KB	Count: 1 209 558	1.861
top-10	-	512 KB	devops: 1 209 558 (highest)	1.850
diff	`error`	256 KB	−495 377 (v1: 605 079, v2: 109 702)	3.636
diff	`request`	256 KB	−247 648 (v1: 302 397, v2: 54 749)	3.762

Table 1: Query results from execution screenshot

Key observations:

- **Case insensitivity verified:** `DEVOPs`, `DEVOPS`, and `devops` all return count 1 209 558.
- **Single-file queries** (word, top) complete in ≈ 1.8–1.9 s with 256 KB buffer.
- **Diff queries** take ≈ 3.6–3.8 s (two sequential file reads).
- **Buffer size impact:** top-10 with 512 KB buffer ran in 1.85 s, comparable to 256 KB runs, indicating I/O is not the bottleneck.
- All results match the reference checker outputs, confirming correctness of the boundary-handling algorithm.

```
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> g++ -O2 -o analyzer  230509_joel.cpp -static-libgcc -static-libstdc++
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file test_logs.txt --version v1 --buffer 256 --query word --word error
Version: v1
Count: 605079
Buffer Size (KB): 256
Execution Time (s): 1.91751
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file test_logs.txt --version v1 --buffer 256 --query word --word DEVOPs
Version: v1
Count: 1209558
Buffer Size (KB): 256
Execution Time (s): 1.80975
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file test_logs.txt --version v1 --buffer 256 --query word --word DEVOPS
Version: v1
Count: 1209558
Buffer Size (KB): 256
Execution Time (s): 1.86146
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file test_logs.txt --version v1 --buffer 512 --query top --top 10
Version: v1
Top 10 words in Version: v1
devops: 1209558
info: 604266
warning: 604149
orderservice: 484437
paymentservice: 484078
authservice: 483842
searchservice: 483162
userservice: 483125
Buffer Size (KB): 512
Execution Time (s): 1.85006
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file1 test_logs.txt --version1 v1 --file2 verbose_logs.txt --version2 v2 --buffer 256 --query diff --word error
Version 1: v1
Count: 605079
Version 2: v2
Count: 109702
Difference (v2 - v1): -495377
Buffer Size (KB): 256
Execution Time (s): 3.63551
PS E:\Volume H\CS253\CPP_Assignment\230509_joel> ./analyzer --file1 test_logs.txt --version1 v1 --file2 verbose_logs.txt --version2 v2 --buffer 256 --query diff --word request
Version 1: v1
Count: 302397
Version 2: v2
Count: 54749
Difference (v2 - v1): -247648
Buffer Size (KB): 256
Execution Time (s): 3.76206
PS E:\Volume H\CS253\CPP_Assignment\230509_joel>
```

Figure 1: Terminal output - compilation and all query types